Movate Python Backend Developer

Interview Preparation Set — 50 Questions

Target: 1–3 years experience | Python Backend Developer
Focus: Python, Coding, DSA, OOP, REST APIs, SQL, Async/Concurrency, Microservices & Kafka

1. Python Fundamentals & Coding — Very High Priority

What are the main features of Python?

What is the difference between list, tuple, set, and dictionary?

What is the difference between mutable and immutable objects?

What is the difference between == and is?

Explain shallow copy vs deep copy.

What are *args and **kwargs?

What are list, dictionary, and set comprehensions?

What are lambda functions? Give a practical example.

What are map(), filter(), and reduce()?

What are iterators and generators? What is the use of yield?

Reverse a string without using [::-1].

Check whether a string is a palindrome.

Find the largest and second-largest number in a list.

Find duplicate elements in a list.

Remove duplicates while preserving the original order.

Count the frequency of each element in a list.

Find the first non-repeating character in a string.

Check whether two strings are anagrams.

Move all zeroes to the end of an array.

Find two numbers whose sum equals a given target (Two Sum).

2. DSA & Problem Solving — High Priority

Find the missing number in an array containing numbers from 1 to n.

Find common elements between two lists.

Find the union and intersection of two arrays.

Find the maximum subarray sum.

Implement binary search.

Implement a sorting algorithm without using sort().

Merge two sorted lists.

Rotate an array by k positions.

Implement a stack using a Python list.

Check whether brackets are balanced using a stack.

3. OOP & Advanced Python — Very High Priority

Explain the four pillars of OOP.

What is the difference between a class and an object?

Explain inheritance with a practical example.

What is method overriding?

What is the difference between @staticmethod, @classmethod, and instance methods?

What are __init__, __str__, and __repr__?

What are decorators? Write a simple custom decorator.

What is exception handling in Python?

How do you create custom exceptions?

Explain Python's GIL. How does it affect multithreading?

4. Backend / REST API — Extremely High Priority

What is a REST API? Explain REST principles.

Explain the difference between GET, POST, PUT, PATCH, and DELETE.

Explain common HTTP status codes: 200, 201, 400, 401, 403, 404, 409, and 500.

How would you create a REST API using FastAPI or Flask?

How do you validate request data in FastAPI?

What is middleware? Give a practical example.

How would you implement JWT authentication in a Python API?

How would you handle exceptions and errors globally in an API?

An API is taking 5 seconds to respond. How would you debug and optimize it?

Design a scalable Python backend API. Explain the role of API Gateway, FastAPI/Flask services, database, cache, Kafka, and background workers.

Important Follow-Up Topics

SQL — High Priority

• What is a JOIN? INNER JOIN vs LEFT JOIN?

• What is an index and when should you use one?

• What is normalization?

• Write a query to find the second-highest salary.

• How do you find duplicate records?

• Explain transactions and ACID properties.

• How would you optimize a slow SQL query?

Async / Concurrency — High Priority

• What are async and await?

• What is an event loop?

• What is a coroutine?

• Threading vs multiprocessing?

• CPU-bound vs I/O-bound tasks?

• When would you use asynchronous programming?

• What is a race condition and how can it be prevented?

Kafka — Medium/High Priority

• What is Apache Kafka?

• What is the difference between a producer and consumer?

• What is a topic?

• What is a partition?

• What is a consumer group?

• What is an offset?

• Why would you use Kafka instead of REST communication in some systems?

Microservices — Medium/High Priority

• Monolith vs microservices?

• What are the advantages and disadvantages of microservices?

• How do microservices communicate?

• What is an API Gateway?

• How do you handle failures between services?

Good-to-Have: Docker / AWS / Kubernetes / Git

• What is Docker? Explain image vs container.

• What is a Dockerfile?

• What is Kubernetes and what is a Pod?

• What are basic AWS services used in backend systems?

• What is CI/CD?

• What Git commands do you use in daily development?

Coding Interview Strategy

For every coding problem, be ready to explain:
1. Approach and logic
2. Why you selected the data structure
3. Time complexity
4. Space complexity
5. Edge cases
6. Possible optimization

Recommended first 15 coding problems: Reverse string, palindrome, largest element, second-largest, duplicates, remove duplicates, frequency counter, anagram, first non-repeating character, Two Sum, missing number, move zeroes, maximum subarray, binary search, balanced parentheses.

Movate Preparation Priority

Area	Priority
Python fundamentals	★★★★★
Python coding / problem solving	★★★★★
REST API / Backend	★★★★★
OOP	★★★★★
SQL	★★★★■
Async programming	★★★★■
Multithreading / Multiprocessing	★★★★■
FastAPI / Flask / Django	★★★★■
Microservices	★★★★■
Kafka	★★★■■
Docker / Git / CI-CD	★★★■■
Kubernetes / AWS / MongoDB	★★■■■
Advanced DSA	★★■■■

Key advice: For a 1–3 year Python Backend Developer role, do not prepare only advanced DSA. Be strong in Python fundamentals, practical coding, OOP, REST APIs, SQL, and the concurrency topics explicitly listed in the JD. Interviewers may take one simple coding problem and then ask you to explain, optimize, test, and convert the logic into an API.